Algorithms for Data Science

Lecture #9: Limits of Efficient Computation

Mateusz Buczynski

University of Warsaw

27.05.2026

Thanks to dr Krzysztof Fleszar for the base material.

A Core Algorithmic Question

Given an algorithm, ask

Can we do better?

Two possible outcomes

- yes: there is room for asymptotic improvement
- no: a lower bound blocks that improvement

Lower Bounds Tell Us What Is Impossible

Meaning

A lower bound of $\Omega(f(n))$ means that every algorithm needs at least that much time in the chosen model.

Decision rule

If our algorithm already matches the lower bound asymptotically, then it is optimal in that model.

Sorting as the Familiar Example

Comparison model

Every comparison-based sorting algorithm needs

$$\Omega(n \log n)$$

comparisons in the worst case.

Consequence

No comparison sort can be worst-case $O(n)$.

Decision Problems

Format

The answer must be only

`YES` or `NO`.

Examples

- Is there a path from s to t?
- Is there a knapsack solution of value at least K?
- Is a Boolean formula satisfiable?

Example: Is There a Path from s to t?

Input

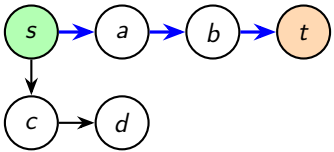
A directed graph G, a source s, and a target t.

Question

Is there a directed path from s to t?

Answer

`YES` or `NO` — nothing else.



Path $s \to a \to b \to t$ exists: `YES`.

Formal View: Is $w \in S$?

Encoding

Every instance (graph, numbers, formula, ...) is encoded as a string w over some alphabet.

Decision problem as a language

A decision problem is identified with a language

$$S \subseteq \Sigma^*$$

the set of all encodings of YES-instances.

The canonical form

Given input w, decide: $w \in S$?

Path problem

$S = \{ \langle G, s, t \rangle \mid G \text{ has a path from } s \text{ to } t \}$

Why This Formulation?

- **Why yes/no problems?**
  - simpler to describe theoretically
  - decision version is at least as hard as the optimization version
- **Why encode input as a word?**
  - gives a precise, model-independent definition
  - input size = number of letters, so complexity is well-defined

Time Hierarchy

Time Hierarchy Theorem

Let $f(n)$, $g(n)$ be running time functions.

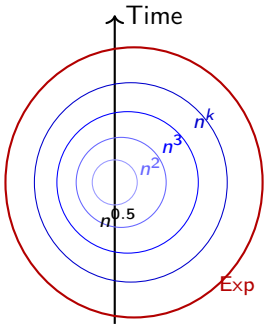
If g grows faster than f (significantly),
then there is a problem solvable
in $O(g(n))$ but not in $O(f(n))$.

Corollary 1

For every k, there is a problem solvable
in polynomial time but not in $O(n^k)$.

Corollary 2

There is a problem solvable in exponential time
but not in polynomial time.



Class P

Definition

P is the class of decision problems solvable in polynomial time.

Interpretation

These are the problems we usually call efficiently solvable.

Examples

Sorting, shortest paths, MST, bipartite matching, linear programming.

Polynomial in What?

Important detail

Running time must be polynomial in the input length, not just in a numeric parameter that appears inside the input.

Why this matters

The number $M = 10^9$ is huge, but its binary encoding length is only about 30 bits.

Knapsack and Pseudo-Polynomial Time

Reminder from DP

The standard DP for knapsack runs in

$$\Theta(nM).$$

But

This is polynomial in the numeric capacity M, not in the bit-length of M.

Name

Such running time is called pseudo-polynomial.

Knapsack: Is $\Theta(nM)$ Polynomial in the Input?

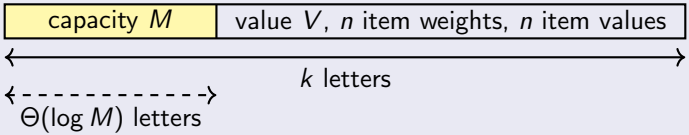
KNAPSACK believed not to be in P

Can one pack items into the knapsack of total value at least V?

Input: capacity M, value V, n item weights, n item values

(capacity M: highlighted)

Input encoding has k letters total



Key fact

To write any integer M, $\Theta(\log M)$ letters are needed and suffice.
(e.g. $M = 1000$ uses $4 = 1 + \log_{10} 1000 = \Theta(\log M)$ digits)

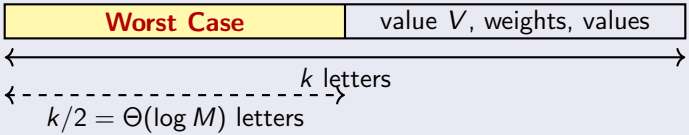
Knapsack: No, It Is Exponential

KNAPSACK believed not to be in P

Can one pack items into the knapsack of total value at least V?

Input: $\underbrace{\text{capacity } M}_{\text{highlighted}}$, value V, n item weights, n item values

Worst-case input: M dominates



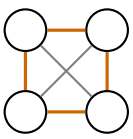
$$\log M = \Theta(k) \implies M = 2^{\Theta(k)}$$

$$\Theta(n \cdot M) = 2^{\Omega(k)} \quad \Rightarrow \quad \textbf{No, exponential!}$$

Hamiltonian Cycle: Believed Not in P

Hamiltonian Cycle

Is there a cycle visiting each vertex exactly once?



Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known

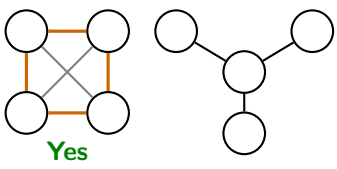
Hamiltonian Cycle: Believed Not in P

Hamiltonian Cycle

Is there a cycle visiting each vertex exactly once?

Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known



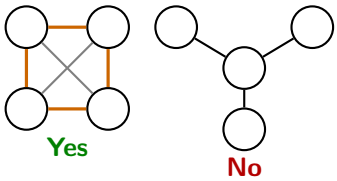
Hamiltonian Cycle: Believed Not in P

Hamiltonian Cycle

Is there a cycle visiting each vertex exactly once?

Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known

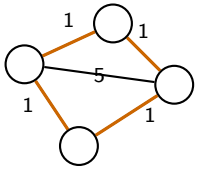


Traveling Salesman: Believed Not in P

Traveling Salesman (Decision)

Is there a tour visiting all vertices of total cost $\leq C$?

(vertices may be revisited; must return to start)



Status

- NP-complete
- no polynomial-time algorithm is known
- approximable within 3/2 in metric case (Christofides)

Traveling Salesman: Believed Not in P

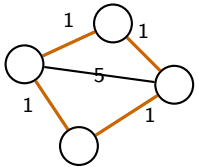
Traveling Salesman (Decision)

Is there a tour visiting all vertices of total cost $\leq C$?

(vertices may be revisited; must return to start)

Status

- NP-complete
- no polynomial-time algorithm is known
- approximable within 3/2 in metric case (Christofides)



Tour cost: $1+1+1+1 = 4$. $C = 5$? **Yes**.

SAT: Believed Not in P

Boolean Satisfiability (SAT)

Is there a true/false assignment making the formula true?

Example formula

$$(a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg b)$$

SAT: Believed Not in P

Boolean Satisfiability (SAT)

Is there a true/false assignment making the formula true?

Example formula

$$(a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg b)$$

Assignment $a = \top,\ b = \bot$: $(\top \vee \bot) \wedge (\bot \vee \top) \wedge (\top \vee \top) = \top$. **Yes.**

Status

- first proven NP-complete (Cook–Levin, 1971)
- best known exact algorithms: $O(2^n)$ in the number of variables
- no polynomial-time algorithm is known

Polynomial-Time Reductions

Main idea

To show that problem A is no harder than problem B, transform instances of A into instances of B.

Notation

$$A \leq_p B$$

means A reduces to B in polynomial time.

Why Reductions Are Powerful

If

$$A \leq_p B$$

and B has a polynomial-time algorithm,

then

A also has a polynomial-time algorithm.

Interpretation

An algorithm for B can be reused as a black box for A.

Reduction Direction Matters

Warning

$$A \leq_p B$$

does *not* mean B reduces to A.

How to read it

If B is easy, then A is easy. If A is hard, then B must be at least that hard.

Example Reduction: Ham. Cycle $\leq_p$ TSP

Construction

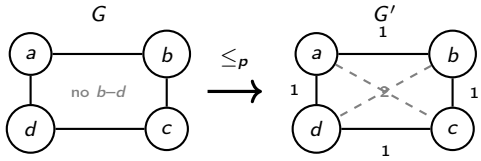
Given graph $G = (V, E)$, build a complete weighted graph G':

$$w(u, v) = \begin{cases} 1 & \{u, v\} \in E \\ 2 & \{u, v\} \notin E \end{cases}$$

Ask TSP: is there a tour in G' of cost $\leq |V|$?

Correctness

G has a Ham. cycle $\iff$ G' has a tour of cost $\leq |V|$.



Tour $a \to b \to c \to d \to a$ costs $4 = |V|$. **Yes.**

Consequence

If TSP $\in P$ then Ham. Cycle $\in P$.

A Canonical Example Family

Classic hard decision problems

- SAT
- Hamiltonian Cycle
- Traveling Salesman (decision)
- KNAPSACK (decision)

The big surprise

These problems are connected by reductions; they are not isolated accidents.

Class *NP*

Meaning

NP stands for *nondeterministic polynomial time*.

Practical intuition

A `YES` answer has a certificate that can be checked in polynomial time.

What Does "Easy to Verify" Mean?

SAT

Certificate: an assignment of truth values.

Knapsack

Certificate: a proposed subset of items.

Common theme

Checking a proposed solution is much easier than searching for one.

Containment

Basic fact

$$P \subseteq NP$$

Why

If we can solve a problem quickly, then we can certainly verify a proposed answer quickly.

NP-Hard and NP-Complete

NP-hard

At least as hard as every problem in NP.

NP-complete

In NP and NP-hard.

Interpretation

NP-complete problems are the hard core inside NP.

Why NP-Complete Problems Matter So Much

Key implication

If one NP-complete problem has a polynomial-time algorithm, then

$$P = NP.$$

Therefore

A breakthrough on one canonical NP-complete problem would propagate to all problems in NP.

The P vs NP Question

One of the biggest open problems in mathematics and computer science

$$P \stackrel{?}{=} NP$$

Two worlds

- if $P = NP$: efficient exact algorithms exist for all NP problems
- if $P \neq NP$: some efficiently verifiable problems are inherently hard to solve exactly

How to Think in Practice

For NP-complete problems

Do not expect a general exact polynomial-time algorithm unless a major theoretical breakthrough happens.

So what do we do?

Use structure, special cases, approximation, heuristics, randomization, or parameterized methods.

Video

https://www.youtube.com/watch?v=x36UmiSiEzc

Approximation Algorithms

Idea

Return a solution that may be non-optimal, but comes with a provable quality guarantee.

Why useful

When exact optimization is too slow, a guaranteed near-optimal solution can still be very valuable.

Approximation vs Heuristics

Approximation

Has a formal bound on solution quality.

Heuristic

Often fast and useful, but no provable guarantee is required.

Metric TSP Approximation: Setup

Optimization problem

Visit all cities and return to the start with minimum total cost.

Assumption for approximation

We work in the metric case, where the triangle inequality holds.

Why the MST Helps for TSP

Key inequality

If T is a minimum spanning tree and OPT is the optimal TSP tour, then

$$\text{cost}(T) \leq \text{cost}(OPT).$$

Reason

Removing one edge from any TSP tour leaves a spanning tree.

A 2-Approximation for Metric TSP

Algorithm

1. compute an MST
2. double its edges to get a closed walk
3. shortcut repeated vertices

Guarantee

$$\text{cost}(\text{tour}) \leq 2 \cdot \text{cost}(OPT).$$

Why the 2-Approximation Bound Works

Step 1

$$\text{cost}(T) \leq \text{cost}(OPT)$$

Step 2

Doubling tree edges gives cost

$$2 \cdot \text{cost}(T).$$

Step 3

Shortcutting does not increase cost in metric spaces.

What Approximation Gives Us

- a principled response to NP-hard optimization
- predictable quality
- scalable practical algorithms in many settings

Data-science relevance

Large optimization problems often need good answers fast, not perfect answers slowly.

What to Remember

- Lower bounds explain when improvement is impossible.
- P means polynomial-time solvable; NP means polynomial-time verifiable for YES answers.
- Reductions transfer tractability and hardness.
- NP-complete problems are the central hard problems inside NP.
- Approximation is a rigorous way to cope with hard optimization problems.

Next Time

Coming next

We step from complexity theory into modern machine learning optimization: gradient descent and training neural networks.